

Parte 1 — O Corpo: o robô autônomo

Onde estamos na jornada. Esta é a primeira parte prática do projeto. Vamos montar um robô capaz de andar sozinho, detectar obstáculos com um sensor de ultrassom e desviar deles — tudo controlado por um microcontrolador ESP32. Primeiro simulamos tudo no computador (sem gastar um parafuso); depois montamos o robô físico, fio por fio, confirmando cada componente antes de ligar.

1.1 O que vamos construir

Um **robô de duas rodas e modo exploração**: ele anda em direções aleatórias pelo ambiente, e quando detecta um obstáculo à frente, para, olha para os dois lados com o sensor (graças ao servo que o gira) e escolhe o caminho mais livre.

COMPORTAMENTO DO ROBÔ

1. Anda em direção aleatória

2. Sensor detecta obstáculo próximo?

SIM → para → recua → olha lados
→ escolhe o lado mais livre
→ vira → continua

NÃO → continua andando

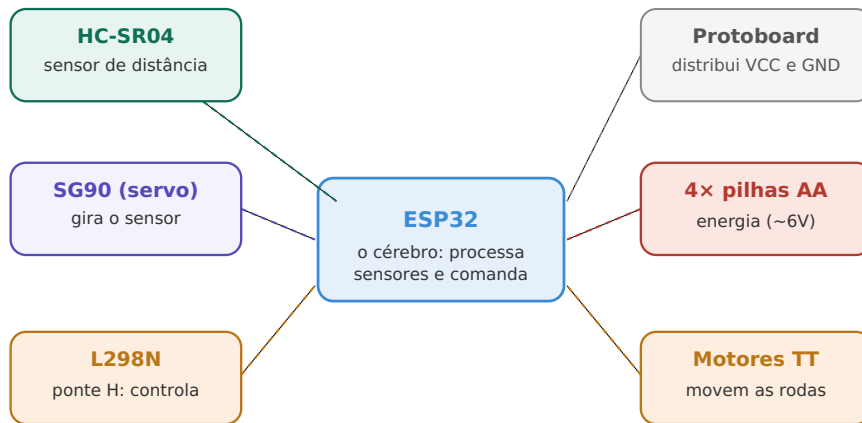
3. Após algum tempo, muda de direção (exploração)

1.2 Lista de componentes

Tudo que você precisa para montar o robô:

Qtd.	Componente	Para que serve
1	ESP32-WROOM-32 DevKit	O microcontrolador (o "cérebro" do corpo)
1	HC-SR04	Sensor de distância ultrassônico
1	SG90	Servo motor — gira o sensor para os lados
1	L298N	Ponte H — controla os dois motores
2	Motor DC TT com caixa de redução	Movem as rodas
2	Rodas compatíveis com motor TT	—
1	Roda boba (castor)	Terceiro ponto de apoio
1	Chassi de acrílico 2WD	A estrutura do robô
1	Suporte para 4x pilhas AA	Fonte de energia
4	Pilhas AA	Energia (~6V com alcalinas)
1	Protoboard	Distribuir VCC e GND
—	Fios jumper Dupont (macho-macho, macho-fêmea)	Conexões
1	Chave liga/desliga (opcional)	—

Os componentes do robô e o que cada um faz



Todos os fios pretos (GND) precisam estar unidos — é a regra de ouro elétrica.

Os componentes do robô e o que cada um faz

Não compre tudo de uma vez. Se quiser testar antes de investir, siga a Seção 1.4 (simulação no Wokwi) — você roda o comportamento completo do robô no computador sem nenhuma peça. Se gostar, aí compra os componentes para a montagem.

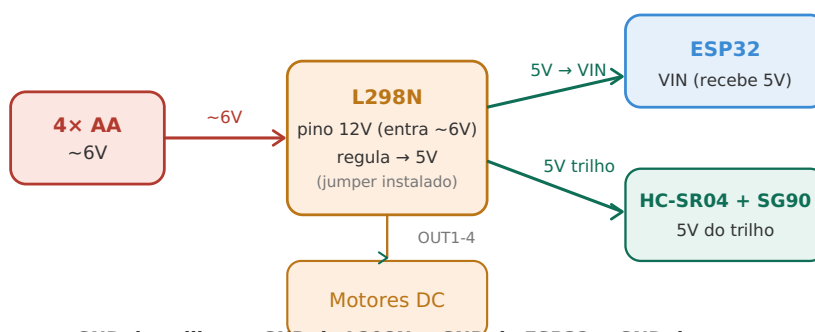
1.3 Entendendo a alimentação antes de montar qualquer fio

Antes de qualquer coisa, é preciso entender como o circuito será alimentado. Errar a alimentação é a causa número um de peças queimadas.

Regra de ouro: todos os componentes do robô precisam compartilhar o mesmo **GND** (terra/negativo). Se o GND de um componente não estiver conectado ao GND do ESP32, os componentes "não falam a mesma língua elétrica" e o circuito não funciona.

O fluxo de energia é este:

Esquema de alimentação: um conjunto de pilhas abastece tudo



GND das pilhas = GND do L298N = GND do ESP32 = GND dos sensores

Todos os terras unidos — sem isso, o circuito não funciona.

Esquema de alimentação

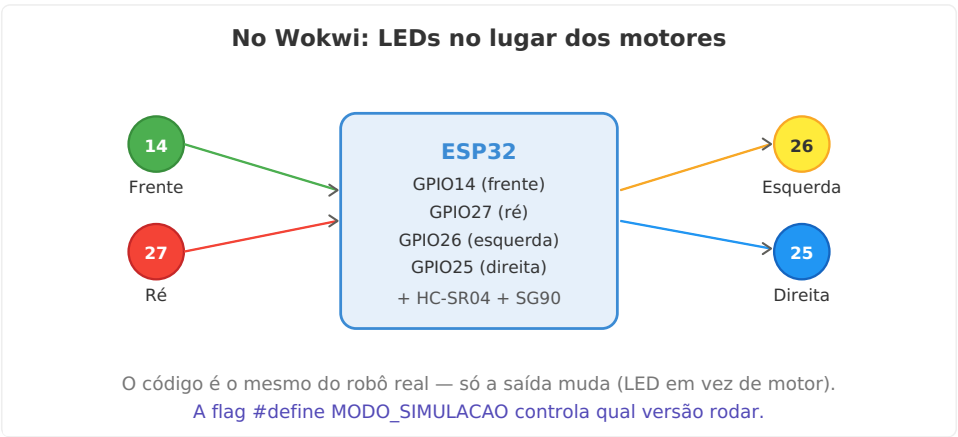
1. As **4 pilhas AA** (~6V) ligam no pino **12V do L298N** (o L298N aceita até 12V; 6V funcionam bem).
2. O **L298N tem um regulador interno de 5V** — o jumper pequeno (que vem instalado de fábrica) precisa estar colocado para ele funcionar. Esse regulador gera 5V estáveis a partir dos 6V das pilhas.
3. Esses **5V saem pelo pino "5V" do L298N** e entram no **pino VIN do ESP32**. Assim as pilhas alimentam também o ESP32.
4. O **ESP32 (via USB no computador)** alimenta o HC-SR04 e o SG90 pelo pino 3V3 ou pelo trilho da protoboard.

Curiosidade: o L298N remove dois jumpers de controle (ENA e ENB) para permitir que o ESP32 controle a velocidade. Se esses jumpers estiverem colocados, os motores ficam sempre na velocidade máxima, sem controle.

1.4 Simulação no Wokwi (sem peça nenhuma)

O **Wokwi** (wokwi.com) é um simulador gratuito e online para ESP32 e Arduino. Você escreve o código, monta o circuito no navegador e roda tudo sem precisar de nenhum componente físico. É perfeito para aprender e testar antes de montar.

O problema com a simulação de motores: o Wokwi não tem um componente de motor DC que responda exatamente como o L298N real. A solução que usaremos: substituímos os motores por **4 LEDs** — cada LED representa uma direção de movimento:



LEDs no lugar dos motores no Wokwi

LED	Cor	GPIO	Significa
Frente	Verde	14	Os motores giram para frente
Ré	Vermelho	27	Os motores giram para trás
Esquerda	Amarelo	26	Virando à esquerda
Direita	Azul	25	Virando à direita

O código é **exatamente o mesmo** do robô real — só a camada de saída muda (LED em vez de motor). Uma única linha controla isso: `#define MODO_SIMULACAO`.

Montando o Wokwi

- Acesse **wokwi.com**, clique em **New Project** e escolha **ESP32**.
- Na aba `diagram.json`, substitua todo o conteúdo por:

```
{
  "version": 1,
  "author": "Robô Autônomo ESP32",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-v1", "id": "esp", "top": 100, "left": 200, "attrs": {} },
    { "type": "wokwi-breadboard-half", "id": "bb", "top": 400, "left": 0, "attrs": {} },
    { "type": "wokwi-hc-sr04", "id": "ultra", "top": -60, "left": 80, "attrs": {} },
    { "type": "wokwi-servo", "id": "servo", "top": -60, "left": 350, "attrs": {} },
    { "type": "wokwi-led", "id": "ledF", "top": 420, "left": 100, "attrs": { "color": "green" } },
    { "type": "wokwi-led", "id": "ledT", "top": 420, "left": 180, "attrs": { "color": "red" } },
    { "type": "wokwi-led", "id": "ledE", "top": 420, "left": 260, "attrs": { "color": "yellow" } },
    { "type": "wokwi-led", "id": "ledD", "top": 420, "left": 340, "attrs": { "color": "blue" } },
    { "type": "wokwi-resistor", "id": "r1", "top": 460, "left": 100, "attrs": { "value": "220" } },
    { "type": "wokwi-resistor", "id": "r2", "top": 460, "left": 180, "attrs": { "value": "220" } },
    { "type": "wokwi-resistor", "id": "r3", "top": 460, "left": 260, "attrs": { "value": "220" } },
    { "type": "wokwi-resistor", "id": "r4", "top": 460, "left": 340, "attrs": { "value": "220" } }
  ],
  "connections": [
    ["esp:3V3", "bb:tp.1", "red", []],
    ["esp:GND.1", "bb:tn.1", "black", []],
    ["bb:tp.2", "ultra:VCC", "red", []],
    ["bb:tn.2", "ultra:GND", "black", []],
    ["esp:D32", "ultra:TRIG", "blue", []],
    ["esp:D33", "ultra:ECHO", "green", []],
    ["bb:tp.3", "servo:V+", "red", []],
    ["bb:tn.3", "servo:GND", "black", []],
    ["esp:D13", "servo:PWM", "orange", []],
    ["esp:D14", "r1:1", "green", []],
    ["r1:2", "ledF:A", "green", []],
    ["esp:D15", "r2:1", "red", []],
    ["r2:2", "ledT:A", "red", []],
    ["esp:D16", "r3:1", "yellow", []],
    ["r3:2", "ledE:A", "yellow", []],
    ["esp:D17", "r4:1", "blue", []],
    ["r4:2", "ledD:A", "blue", []]
  ]
}
```

```

["ledF:C",      "bb:tn.4",  "black",  []],
["esp:D27",     "r2:1",     "red",    []],
["r2:2",        "ledT:A",   "red",    []],
["ledT:C",      "bb:tn.5",  "black",  []],
["esp:D26",     "r3:1",     "yellow", []],
["r3:2",        "ledE:A",   "yellow", []],
["ledE:C",      "bb:tn.6",  "black",  []],
["esp:D25",     "r4:1",     "blue",   []],
["r4:2",        "ledD:A",   "blue",   []],
["ledD:C",      "bb:tn.7",  "black",  []]
]
}

```

1. Na aba **sketch.ino**, cole o código da próxima seção com `#define MODO_SIMULACAO true`.
2. No **Library Manager** (ícone de livro), instale **ESP32Servo**.
3. Clique em ► **Play**.

1.5 O código do robô (unificado — serve para Wokwi e físico)

O código abaixo tem uma única flag que controla tudo. Para o Wokwi: `true`. Para o robô real: `false`. Nada mais muda.

```

/*
=====
ROBÔ AUTÔNOMO - MODO EXPLORAÇÃO
ESP32 + HC-SR04 + SG90 + L298N

Para o Wokwi (simulação com LEDs):
#define MODO_SIMULACAO true

Para o robô físico (L298N + motores):
#define MODO_SIMULACAO false
=====
*/

#include <ESP32Servo.h>

// =====
// MUDE APENAS ESTA LINHA PARA ALTERNAR ENTRE OS DOIS MODOS
// =====
#define MODO_SIMULACAO true

// Pinos do sensor e do servo (iguais nos dois modos)
const int TRIG_PIN = 32;
const int ECHO_PIN = 33;
const int SERVO_PIN = 13;

// Pinos do L298N (usados só no robô físico)
#if !MODO_SIMULACAO
    const int ENA = 23, IN1 = 22, IN2 = 21;
    const int ENB = 5,  IN3 = 19, IN4 = 18;
#endif

// Pinos dos LEDs (usados só no Wokwi)
#if MODO_SIMULACAO
    const int LED_FRENTE = 14;
    const int LED_TRAS = 27;
    const int LED_ESQUERDA = 26;
    const int LED_DIREITA = 25;
#endif

// Configurações gerais
const int DISTANCIA_OBSTACULO = 25;
const int VELOCIDADE_MINIMA = 140;
const int VELOCIDADE_MAXIMA = 220;
const int VELOCIDADE_RE = 160;
const int VELOCIDADE_CURVA = 190;
const int TEMPO_RECUO = 250;
const int TEMPO_CURVA_OBSTACULO = 450;
const unsigned long DUR_MIN_MOV = 2000;
const unsigned long DUR_MAX_MOV = 6000;
const unsigned long DUR_MIN_CURVA = 300;
const unsigned long DUR_MAX_CURVA = 800;

const int SERVO_CENTRO = 90;

```

```
const int SERVO_ESQUERDA = 30;
const int SERVO_DIREITA = 150;
```

```
Servo sensorServo;
```

```
// Controle do modo exploração
unsigned long inicioMovimento = 0, duracaoMovimento = 0;
int velocidadeAtual = 0, movimentoAtual = 0, ultimaCurva = 0;
```

```
// =====
// MEDIR DISTÂNCIA
// =====
```

```
float medirDistancia() {
    digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);
    long dur = pulseIn(ECHO_PIN, HIGH, 30000);
    if (dur == 0 || dur < 60) return 400;
    return dur * 0.0343 / 2.0;
}
```

```
// =====
// CONTROLE DE MOVIMENTO
// =====
```

```
void desligarLeds() {
    #if MODO_SIMULACAO
        digitalWrite(LED_FRENTE, LOW); digitalWrite(LED_TRAS, LOW);
        digitalWrite(LED_ESQUERDA, LOW); digitalWrite(LED_DIREITA, LOW);
    #endif
}
```

```
void andarFrente(int v) {
    #if MODO_SIMULACAO
        desligarLeds(); digitalWrite(LED_FRENTE, HIGH);
    #else
        digitalWrite(IN1,HIGH); digitalWrite(IN2,LOW);
        digitalWrite(IN3,HIGH); digitalWrite(IN4,LOW);
        analogWrite(ENA,v); analogWrite(ENB,v);
    #endif
}
```

```
void andarTras(int v) {
    #if MODO_SIMULACAO
        desligarLeds(); digitalWrite(LED_TRAS, HIGH);
    #else
        digitalWrite(IN1,LOW); digitalWrite(IN2,HIGH);
        digitalWrite(IN3,LOW); digitalWrite(IN4,HIGH);
        analogWrite(ENA,v); analogWrite(ENB,v);
    #endif
}
```

```
void virarEsquerda(int v) {
    #if MODO_SIMULACAO
        desligarLeds(); digitalWrite(LED_ESQUERDA, HIGH);
    #else
        digitalWrite(IN1,LOW); digitalWrite(IN2,HIGH);
        digitalWrite(IN3,HIGH); digitalWrite(IN4,LOW);
        analogWrite(ENA,v); analogWrite(ENB,v);
    #endif
}
```

```
void virarDireita(int v) {
    #if MODO_SIMULACAO
        desligarLeds(); digitalWrite(LED_DIREITA, HIGH);
    #else
        digitalWrite(IN1,HIGH); digitalWrite(IN2,LOW);
        digitalWrite(IN3,LOW); digitalWrite(IN4,HIGH);
        analogWrite(ENA,v); analogWrite(ENB,v);
    #endif
}
```

```
void parar() {
    #if MODO_SIMULACAO
        desligarLeds();
    #else
        analogWrite(ENA,0); analogWrite(ENB,0);
        digitalWrite(IN1,LOW); digitalWrite(IN2,LOW);
        digitalWrite(IN3,LOW); digitalWrite(IN4,LOW);
    #endif
}
```

```

}

// =====
// VARREDURA COM SERVO
// =====
float medirLado(int angulo) {
    sensorServo.write(angulo); delay(400);
    return medirDistancia();
}

// =====
// DESVIO DE OBSTÁCULO
// =====
void evitarObstaculo() {
    parar(); delay(200);
    andarTras(VELOCIDADE_RE); delay(TEMPO_RECUE);
    parar(); delay(200);
    float esq = medirLado(SERVO_ESQUERDA);
    float dir = medirLado(SERVO_DIREITA);
    sensorServo.write(SERVO_CENTRO); delay(300);
    if (esq > dir) { virarEsquerda(VELOCIDADE_CURVA); }
    else { virarDireita(VELOCIDADE_CURVA); }
    delay(TEMPO_CURVA_OBSTACULO);
    parar(); delay(100);
}

// =====
// MODO EXPLORAÇÃO
// =====
void iniciarNovoMovimento() {
    velocidadeAtual = random(VELOCIDADE_MINIMA, VELOCIDADE_MAXIMA + 1);
    int s = random(100);
    movimentoAtual = (s < 50) ? 0 : (s < 75) ? 1 : 2;
    if (movimentoAtual == 1 && ultimaCurva == 1) movimentoAtual = 2;
    if (movimentoAtual == 2 && ultimaCurva == 2) movimentoAtual = 1;
    if (movimentoAtual != 0) ultimaCurva = movimentoAtual;
    duracaoMovimento = (movimentoAtual == 0)
        ? random(DUR_MIN_MOV, DUR_MAX_MOV + 1)
        : random(DUR_MIN_CURVA, DUR_MAX_CURVA + 1);
    inicioMovimento = millis();
}

// =====
// SETUP E LOOP
// =====
void setup() {
    Serial.begin(115200);
    randomSeed(micros());
    pinMode(TRIG_PIN, OUTPUT); pinMode(ECHO_PIN, INPUT);
    #if !MODO_SIMULACAO
        pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT); pinMode(ENA, OUTPUT);
        pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT); pinMode(ENB, OUTPUT);
    #endif
    #if MODO_SIMULACAO
        pinMode(LED_FRENTE, OUTPUT); pinMode(LED_TRAS, OUTPUT);
        pinMode(LED_ESQUERDA, OUTPUT); pinMode(LED_DIREITA, OUTPUT);
    #endif
    sensorServo.setPeriodHertz(50);
    sensorServo.attach(SERVO_PIN, 500, 2400);
    parar(); sensorServo.write(SERVO_CENTRO); delay(1000);
    iniciarNovoMovimento();
}

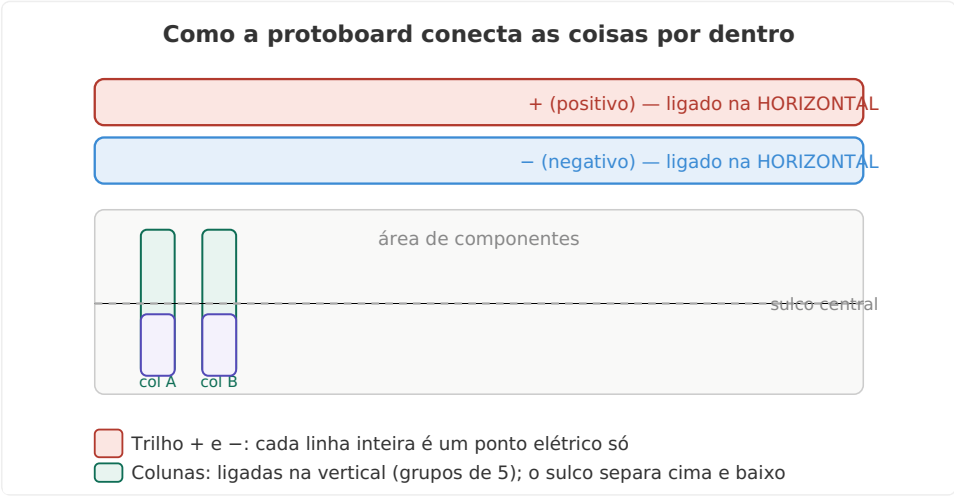
void loop() {
    float dist = medirDistancia();
    if (dist <= DISTANCIA_OBSTACULO) {
        evitarObstaculo();
        iniciarNovoMovimento();
        return;
    }
    if (movimentoAtual == 0) andarFrente(velocidadeAtual);
    else if (movimentoAtual == 1) virarEsquerda(velocidadeAtual);
    else virarDireita(velocidadeAtual);
    if (millis() - inicioMovimento >= duracaoMovimento)
        iniciarNovoMovimento();
    delay(50);
}

```

Para passar do Wokwi para o robô físico: troque a linha `#define MODO_SIMULACAO true` por `#define MODO_SIMULACAO false`. Só isso. Todo o resto do código permanece idêntico.

1.6 Como entender a protoboard

Se você nunca usou uma protoboard, ela parece confusa à primeira vista. A chave é saber como as conexões internas funcionam:



Como a protoboard funciona

Três regras: 1. Os **trilhos** de cima e de baixo (as linhas `+` e `-`) são ligados na **horizontal** — toda a linha é um único ponto elétrico. É aqui que colocamos VCC e GND para distribuir para os componentes. 2. As **colunas** centrais são ligadas na **vertical**, em grupos de 5 furos. É onde você espeta os componentes (cada coluninha de 5 furos é um nó). 3. O **sulco central** separa a metade de cima da metade de baixo — um furo de cima não se conecta ao de baixo mesmo estando alinhados.

Na prática: ligamos um fio do `3V3` do ESP32 no trilho `+` e um fio do `GND` no trilho `-`. Daí, todos os componentes pegam energia e terra do trilho, sem precisar amontoar fios no ESP32.

1.7 Montagem física: passo a passo

Regra de segurança: monte sempre **sem energia** (cabo USB desconectado e pilhas fora do suporte). Só energize na hora de testar, e siga a ordem de energização que indicamos.

Passo 1 — Montagem mecânica

- Monte os **motores TT** no chassi (parafuse ou encaixe conforme o kit).
- Encaixe as **rodas** nos eixos dos motores.
- Fixe a **roda boba** na parte de trás.
- Posicione o **suporte de pilhas** no chassi.
- Monte o **SG90** na frente do chassi, no suporte do sensor.
- Encaixe o **HC-SR04** no braço do servo.

Passo 2 — Trilhos de energia na protoboard

- Fio **vermelho**: pino `3V3` do ESP32 → trilho `+` da protoboard.
- Fio **preto**: pino `GND` do ESP32 → trilho `-` da protoboard.

Passo 3 — HC-SR04

Pino do sensor	Conectar em
VCC	Trilho + (vermelho)
GND	Trilho - (preto)
TRIG	GPIO32 do ESP32

ECHO	GPIO33 do ESP32
------	-----------------

Passo 4 — SG90

Fio do servo	Conectar em
Marrom (GND)	Trilho –
Vermelho (V+)	Trilho +
Laranja (sinal)	GPIO13 do ESP32

Passo 5 — L298N e motores

Pino	Conectar em
IN1	GPIO22 do ESP32
IN2	GPIO21 do ESP32
ENA	GPIO23 do ESP32
IN3	GPIO19 do ESP32
IN4	GPIO18 do ESP32
ENB	GPIO5 do ESP32
GND	Trilho – e negativo das pilhas
12V	Positivo das pilhas (~6V)
5V	VIN do ESP32
OUT1/OUT2	Fios do motor esquerdo
OUT3/OUT4	Fios do motor direito

Confirme: o **jumper de 5V do L298N** está colocado? (pequeno conectorzinho plástico perto do conector de alimentação). Sem ele, o regulador interno não funciona e o ESP32 não recebe energia das pilhas.

Mapa de pinos completo

Mapa de pinos definitivo — ESP32 → componentes

Componente / função	Pino ESP32
HC-SR04 · TRIG (dispara o som)	GPIO 32
HC-SR04 · ECHO (recebe o eco)	GPIO 33
Servo SG90 · PWM (sinal de ângulo)	GPIO 13
L298N · IN1 (motor esq. sentido A)	GPIO 22
L298N · IN2 (motor esq. sentido B)	GPIO 21
L298N · ENA (velocidade motor esq.)	GPIO 23
L298N · IN3/IN4 (motor dir. A/B)	GPIO 19 / 18
L298N · ENB (velocidade motor dir.)	GPIO 5

Mapa de pinos definitivo

Passo 6 — Checagem de segurança (antes de ligar)

Antes de conectar qualquer energia, confirme cada item:

- [] Fio **vermelho** das pilhas no pino **12V** do L298N?
- [] Fio **preto** das pilhas no **GND** do L298N?
- [] **GND do L298N** ligado ao trilho – da protoboard?

- [] Pino **5V do L298N** ligado ao **VIN** do ESP32?
- [] Jumper de 5V do L298N **colocado**?
- [] Os jumpers de **ENA e ENB foram removidos** (se existiam)?
- [] Todos os 6 fios de controle (IN1 a ENB) ligados nos GPIOs certos?
- [] Positivo e negativo das pilhas **não estão se tocando**?

Passo 7 — Ordem de energização

1. **Primeiro:** conecte o USB no ESP32 (para o computador gravar o código).
2. Grave o código com `#define MODO_SIMULACAO false`.
3. **Depois:** coloque as pilhas (ou ligue a chave do suporte).
4. Mantenha o robô com as **rodas no ar** no primeiro teste.

Passo 8 — Se algo não sair como esperado

Sintoma	Provável causa	Solução
Upload falha em "Connecting..."	ESP32 precisa do modo boot	Segure o botão BOOT durante "Connecting..."
Motores não giram	Jumper ENA/ENB presente	Remova os dois jumpers
ESP32 não liga com as pilhas	Jumper de 5V ausente	Confirme e coloque o jumper
Um motor gira ao contrário	Fios do motor invertidos	Troque os dois fios desse motor nos bornes OUT
Sensor não lê distância	TRIG/ECHO trocados	Verifique a ligação nos GPIOs 32 e 33

1.8 Testando por partes (a regra de ouro)

Nunca teste tudo de uma vez. A ordem correta é:

1. Teste do HC-SR04 (só USB, sem pilhas):

```
const int TRIG = 32, ECHO = 33;
void setup() { Serial.begin(115200); pinMode(TRIG,OUTPUT); pinMode(ECHO,INPUT); }
void loop() {
  digitalWrite(TRIG,LOW); delayMicroseconds(2);
  digitalWrite(TRIG,HIGH); delayMicroseconds(10); digitalWrite(TRIG,LOW);
  long d = pulseIn(ECHO,HIGH,30000);
  Serial.println(d ? String(d*0.0343/2.0) + " cm" : "sem eco");
  delay(300);
}
```

Aproxime a mão do sensor — os números devem mudar. ✓

2. Teste do servo (só USB):

```
#include <ESP32Servo.h>
Servo s;
void setup() { s.setPeriodHertz(50); s.attach(13, 500, 2400); }
void loop() { s.write(30); delay(1500); s.write(90); delay(1500); s.write(150); delay(1500); }
```

O braço deve girar nas 3 posições. ✓

3. Teste dos motores (USB + pilhas, rodas no ar):

```
void setup() {
  int pinos[] = {22,21,23,19,18,5};
  for (int p : pinos) pinMode(p, OUTPUT);
}
void loop() {
  Serial.println("frente");
  digitalWrite(22,HIGH); digitalWrite(21,LOW);
  digitalWrite(19,HIGH); digitalWrite(18,LOW);
  analogWrite(23,200); analogWrite(5,200);
  delay(2000);
  analogWrite(23,0); analogWrite(5,0); delay(500);
}
```

Os dois motores devem girar para frente e parar. Confirme que o motor esquerdo gira no lado esquerdo e vice-versa. ✓

Só depois que os três testes passarem, grave o código completo de exploração.

Encerramento da Parte 1

Você construiu um robô autônomo que:

- ✓ Roda no simulador Wokwi (LEDs indicando direção)
- ✓ Roda no hardware real (motores + sensor + servo)
- ✓ Explora o ambiente com movimento aleatório
- ✓ Detecta e desvia de obstáculos autonomamente
- ✓ Usa um único código para os dois modos (só muda uma linha)

Na **Parte 2**, vamos entender o que é uma LLM e por que um modelo de linguagem pode "morar" num microcontrolador. Na **Parte 3**, construímos esse modelo do zero.